ENTREGA 1 – DESPLIEGUE MANUAL DE UN MICROSERVICIO SOBRE UN SERVICIO DE PLATAFORMA COMO SERVICIO (PAAS)

OBJETIVOS

- Desarrollar y desplegar un microservicio que involucre el uso de servidores de nube pública y servicios de control de versiones.
- Realizar el despliegue manual de una aplicación sobre los servicios de un proveedor cloud, **sin incluir** servicios de Continuous Integration / Continuous Deployment.

TIEMPO DE DEDICACIÓN

Esta entrega del proyecto está planeada para dos semanas (1 y 2), en las que cada estudiante deberá invertir las horas definidas en la planeación semanal. Recuerde que para el éxito de la actividad es necesario que conforme un equipo de trabajo según las pautas definidas en el curso.

LECTURAS PREVIAS

- Tutoriales públicos de Flask o aquellos visto en cursos anteriores
- Tutorial de AWS Beanstalk del curso
- Documentación pública de AWS Beanstalk
- AWS Elastic Beanstalk deployment strategies
- Documentación pública de GitHub

ESQUEMA DE EVALUACIÓN

La distribución de la calificación de la entrega se realizará de la siguiente manera:

- Verificación funcional de los requerimientos de la aplicación desplegada sobre los servicios del proveedor de nube, configuración de GitHub y documentación en Postman: **70%**
- Documento del despliegue (más adelante se detalla lo que debe contener este documento): **30%**

LUGAR Y FORMATO DE ENTREGA

La entrega está conformada por:

- Aplicación en ejecución sobre AWS Beanstalk accesible vía Postman
- Repositorio con el código fuente en GitHub. El repositorio debe ser creado por los estudiantes.
- Documento del despliegue realizado

INFRAESTRUCTURA REQUERIDA PARA EL DESPLIEGUE

En este proyecto se planea utilizar la siguiente infraestructura y servicios:

- AWS Academy o cuenta de AWS propia
- Servicios de AWS

RECOMENDACIONES DEL DOCUMENTO DE LA ENTREGA

Se deberá entregar un documento de la entrega. El nombre de este documento debe ser: **"Proyecto 1 entrega 1 – Documento.pdf"**

RECOMENDACIONES Y CONSIDERACIONES

En este proyecto se deberá desarrollar la aplicación Web con las funcionalidades mencionadas. La aplicación web deberá ser desarrollada en el lenguaje de programación **Python** y el framework **FLASK.**

Esta entrega se realiza sobre un proveedor de nube pública, utilizando servicios PaaS.

Se requiere utilizar el siguiente stack tecnológico para el desarrollo de la entrega utilizando las versiones que más se les facilite o las más recientes:

1. **Python**. lenguaje de programación
2. **Flask Framework**: micro framework web.
3. **Flask SQLAlchemy**: una extensión de Flask que agrega soporte para el ORM SQLAlchemy, mapeador relacional de objetos que simplifica la interacción con una base de datos SQL.
4. **Flask RESTful**: una extensión de Flask para desarrollar API REST con un patrón de diseño orientado a objetos.
5. **Flask Marshmallow**: una extensión de Flask que se integra con Marshmallow, una biblioteca de Python para la serialización de objetos.

6. **Flask JWT Extended**: agrega soporte para el uso de JSON Web Tokens (JWT) a Flask para proteger las vistas, creación de tokens a partir de objetos complejos u objeto complejo a partir de tokens recibidos, actualizar tokens, entre otras funciones.

7. **Werkzeug**: biblioteca completa de aplicaciones web WSGI.

8. **PostgreSQL:** Motor open source de base de datos SQL.

ALCANCE DE LA ENTREGA

Una compañía multinacional está interesada en mejorar su proceso de gestión de listas negras de emails dado que en los últimos años ha recibido varias demandas y problemas legales por la mala gestión de sus listas negras de emails en los cientos de aplicaciones que utiliza para su día a día. Para lo anterior la compañía ha decidido que un equipo centralizado despliegue la primera versión de un nuevo microservicio que permita gestionar la lista negra global de la empresa. El objetivo de este microservicio es que cientos de sistemas internos puedan consultar si un email está en la lista negra global de la empresa o no, así como agregar emails a la lista negra global.

DESARROLLO DE LA APLICACIÓN

Para realizar la implementación inicial del microservicio, es necesario desarrollar las siguientes actividades:

1. **(45%)** Desarrollar y exponer un servicio REST (POST) para agregar un email a la lista negra global. Este servicio permite agregar un email a la lista negra global. Para agregar un email a la lista negra global se necesita: el email del cliente a agregar a la lista negra global, el id de la app cliente (es un UUID obligatorio), el motivo por el que se agrega a la lista negra (campo opcional de máximo 255 caracteres). Internamente el microservicio debe guardar la dirección IP desde donde se hace la solicitud y la fecha/hora en la que se realiza la solicitud.
2. **(25%)** Desarrollar y exponer un servicio REST (GET) para consultar si un email está en la lista negra global o no. Este microservicio recibe como parámetro el email del cliente y retorna si ese email está en la lista negra global o no.

Diseño del API REST

En este proyecto deberá crear los siguientes endpoints para su API REST. Esta definición deberá ser respetada a lo largo del desarrollo del proyecto.

Endpoint	/blacklists
Descripción	Permite agregar un email a la lista negra global de la organización.

Método	POST
Retorno	**application/json**. Un mensaje de confirmación notificando si la cuenta pudo o no ser creada.
Parámetros del endpoint	email (String) app_uuid (String) blocked_reason (String)
Parámetros de autorización	Bearer Token.

Endpoint	**/blacklists/<string:email>**
Descripción	Permite saber si un email está en la lista negra global de la organización o no, y el motivo por el que fue agregado a la lista negra.
Método	GET
Retorno	**application/json**. Un valor booleano indicando si está o no el email en la lista negra global y el motivo por el que fue agregado.
Parámetros del endpoint	Ninguno.
Parámetros de autorización	Bearer Token.

La API recibe como parámetro un token de autorización. Por simplicidad pueden definir ese token de forma estática o predefinida para toda la aplicación, sin necesidad de generarlo de forma dinámica.

Todas las anteriores funcionalidades deberán ofrecerse desde un único microservicio. Dado que esta es la primera iteración del producto, otras funcionalidades relacionadas no serán consideradas. No se realizará el diseño de la interfaz de usuario, para validar los servicios cree un conjunto de escenarios de prueba con POSTMAN. Debe persistir la información en un servicio de bases de datos relacionales, se recomienda hacer uso de AWS RDS con el motor de base de datos de su preferencia.

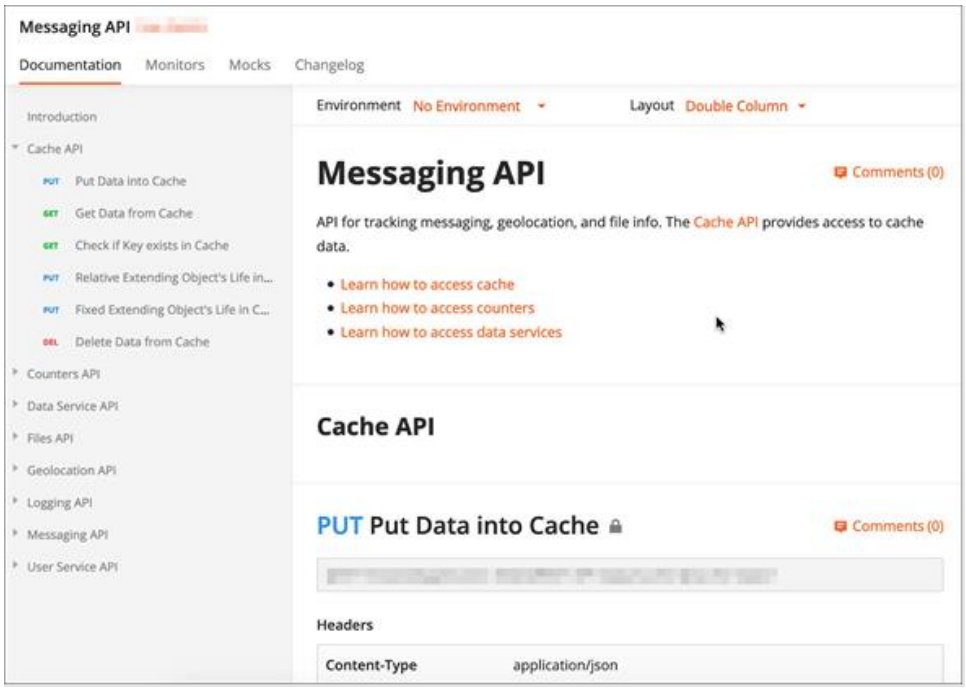
Documentación del API REST

En este proyecto deberá especificar su API y documentar cada uno de sus endpoints. Como no se desarrolla un cliente frontend, utilice Postman para probar los endpoints mediante un conjunto de escenarios de prueba.

Un integrante del equipo debe crear un workspace en Postman (personal o público, según el Plan Free) y una colección con los endpoints, especificando los parámetros mínimos para probar el API. Para compartir el trabajo, el equipo puede invitar a sus integrantes dentro del límite del plan o

exportar la colección y el environment en formato JSON y versionarlos en el repositorio del proyecto.

Documente cada endpoint con su respectivo ejemplo de solicitud y respuesta, y publique la documentación de la colección. La URL publicada debe anexarse al informe de la entrega.



DOCUMENTO DE LA ENTREGA

Elabore un documento que incluya:

1. **(10%)** Capturas de pantalla con el paso a paso de la creación y despliegue de la aplicación sobre AWS Beanstalk (sin o con contenedores). Incluyendo:
    a. Configuración de RDS
    b. Configuración del proyecto en AWS Beanstalk
    c. Configuración de los health checks de la aplicación en AWS Beanstalk

2. **(20%)** Ejecute 4 estrategias de despliegues diferentes en AWS Beanstalk que requieran más de una instancia (por ejemplo all-at-once, rolling, rolling with additional batch, inmutable, traffic-splitting, entre otras). Se recomienda utilizar un grupo de auto scaling de mínimo 3 y máximo 6 instancias. **Documente por cada estrategia** los hallazgos, incluyendo:
    a. Cantidad de instancias utilizadas.
    b. Describa la manera como validó que el despliegue estaba en ejecución.

c. El tiempo total de un despliegue (en minutos y segundos).
d. Si el despliegue se realizó sobre las instancias iniciales o sobre nuevas instancias.
e. Capturas de pantallas de la consola de AWS con los resultados del proceso de despliegue.
f. Hallazgos encontrados de la estrategia de despliegue.

SUSTENTACIÓN DE LA ENTREGA

- **Se debe realizar un video con la sustentación de la entrega.** Este video debe durar máximo 10 minutos (los tutores cortaran el video pasado este tiempo). Lo que se encuentre después del minuto 10 no será teniendo en cuenta y se considera que no fue presentado). El video debe ser explicativo por los integrantes del grupo (uno o varios) y debe contener la explicación y **evidencias (consola de AWS, repositorios del código, etc.) funcionales del trabajo realizado** para cumplir con todos los requisitos descritos en el entregable, sin dejar lugar a dudas de si algo descrito en el entregable fue o no incluido.
- El video debe evidenciar la configuración y el desarrollo de los escenarios de pruebas para los endpoints de la API, los repositorios de código y el ambiente en GitHub y AWS.
- No se contemplará como sustentación, explicar/mostrar en el video, únicamente el contenido del pdf de la entrega ya que este al final será revisado por el tutor.
- La no entrega del video implica que no hay sustentación de la entrega y por ende **la nota de la entrega será calificada con cero (0.0)**.
- Además del video de sustentación, el tutor podrá citar a su grupo a una **sustentación síncrona**.